

CS4084

Quantum Computing

Week 12 – Lecture 22-24

Dr. Maqsood M. Khan

Department of Computer Science
FAST-NUCES Peshawar

Outline

- 1 Bridging Foundations to Algorithms
- 2 Connection: Why the Review Matters Today
- 3 Bernstein-Vazirani Problem
- 4 Simon's Algorithm
- 5 Lecture Summary: The Quantum Model & Quantum Speedup

Quick Review: From Circuits to Algorithms I

- **Quantum Circuits & Wires:** Graphical model where **Wires** represent Qubits and **Gates** represent Unitary operations.
- **Global Operations:** Multi-qubit gates like **CNOT** enable **entanglement**, which is the "fuel" for quantum speedup.
- **Geometry & Projection:** $\langle \phi | \psi \rangle$ measures overlap, and $P = |\psi\rangle\langle\psi|$ is used to extract probabilities—essential for reading algorithm results.
- **Compound Systems ($\otimes$):** Operations combine via the **Kronecker product**, creating the large $(\mathbb{C}^2)^{\otimes n}$ space needed for complex queries.
- **New Focus: The Query Model of Computation**
 - **The Query Gate (O_f):** A "Black Box" (Oracle) that evaluates a function $f(x)$ in a single step.
 - **Standard Picture:** We compare how many "queries" a classical computer needs versus a quantum computer to solve a specific problem.
- **Quantum Speedup Examples:**

Quick Review: From Circuits to Algorithms II

- **Deutsch's Algorithm:** Determining if a function is constant or balanced using only **one** query.
- **Phase Kickback:** A clever trick where the output of a query is "kicked" back into the phase of the input qubit—this is the secret to Deutsch-Jozsa.
- **Deutsch-Jozsa & Bernstein-Vazirani:** Solving global function properties faster than any classical "bit-by-bit" check.
- **Simon's Algorithm:** Finding a hidden pattern (s) in a function; the first proof of exponential speedup over classical difficulty.

Connection: Why the Review Matters Today

The "Why"

We use the **Tensor Products** from last week to create **Superposition** ($H^{\otimes n}$). This allows us to query the Oracle with all possible inputs at once!

- **Last Week:** We learned how to write $|00\rangle + |01\rangle + |10\rangle + |11\rangle$.
- **Today:** We send that state through a **Query Gate** to solve **Deutsch's Problem** or **Simon's Problem**.
- **Post-Processing:** We use our knowledge of **Measurement** to turn those quantum amplitudes back into classical answers.

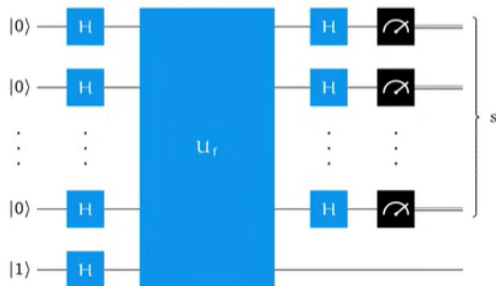
Bernstein-Vazirani Problem

Bernstein-Vazirani problem

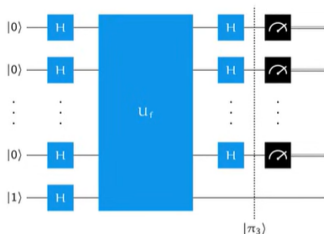
Input: $f : \Sigma^n \rightarrow \Sigma$

Promise: there exists a binary string $s = s_{n-1} \cdots s_0$ for which
 $f(x) = s \cdot x$ for all $x \in \Sigma^n$

Output: the string s



Continue..



$$|\pi_3\rangle = |-\rangle \otimes \frac{1}{2^n} \sum_{y \in \Sigma^n} \sum_{x \in \Sigma^n} (-1)^{f(x)+x \cdot y} |y\rangle$$

π_3

$$\begin{aligned} |\pi_3\rangle &= |-\rangle \otimes \frac{1}{2^n} \sum_{y \in \Sigma^n} \sum_{x \in \Sigma^n} (-1)^{f(x)+x \cdot y} |y\rangle \\ &= |-\rangle \otimes \frac{1}{2^n} \sum_{y \in \Sigma^n} \sum_{x \in \Sigma^n} (-1)^{s \cdot x + y \cdot x} |y\rangle \\ &= |-\rangle \otimes \frac{1}{2^n} \sum_{y \in \Sigma^n} \sum_{x \in \Sigma^n} (-1)^{(s \oplus y) \cdot x} |y\rangle \\ &= |-\rangle \otimes |s\rangle \end{aligned}$$

The Deutsch-Jozsa circuit therefore solves the Bernstein-Vazirani problem with a single query.

Any probabilistic algorithm must make at least n queries to find s .

$|-\rangle$ state for the bottom qubit
and $|s\rangle$ state for the top n qubits

Simon's Algorithm: Finding Hidden Patterns

's algorithm provides the first provable **exponential speedup** over classical computing by finding a secret hidden period s in a function.

- **What it is:** A "Hidden String" problem. The function $f(x)$ repeats itself only when shifted by a secret value s .
- **The Quantum Strategy:**
 - Uses the same **Query Gate** (O_f) and "Hadamard sandwich" structure as Deutsch-Jozsa.
 - Instead of checking inputs one-by-one, it uses interference to sample the global "shape" of the function.
- **The Connection:**
 - Like previous algorithms, it relies on **Phase Kickback** and superposition.
 - It bridges the gap between simple "yes/no" problems and complex "structure-finding" problems like Shor's Algorithm.

Key Advantage

classical computers hit an "Exponential Wall" trying to find this string. Simon's Algorithm bypasses it, using only a linear number of queries.

Simons Problem

Simon's problem

Input: A function $f : \Sigma^n \rightarrow \Sigma^m$

Promise: There exists a string $s \in \Sigma^n$ such that

$$[f(x) = f(y)] \Leftrightarrow [(x = y) \text{ or } (x \oplus s = y)]$$

for all $x, y \in \Sigma^n$

Output: The string s

Case 1: $s = 0^n$

The condition in the promise simplifies to

$$[f(x) = f(y)] \Leftrightarrow [x = y]$$

This is equivalent to f being *one-to-one*.

Case-2

Case 2: $s \neq 0^n$

The function f must be *two-to-one* to satisfy the promise:

$$f(x) = f(x \oplus s)$$

with *distinct output strings* for each pair.

Continue..

Case 2: $s \neq 0^n$

The function f must be *two-to-one* to satisfy the promise:

$$f(x) = f(x \oplus s)$$

with *distinct output strings* for each pair.

x	$f(x)$
000	10011
001	00101
010	00101
011	10011
100	11010
101	00001
110	00001
111	11010

$$s = 011$$

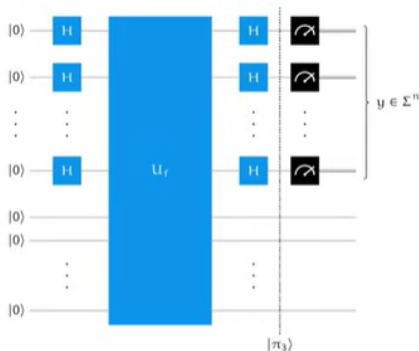
$$f(000) = f(011) = 10011$$

$$f(001) = f(010) = 00101$$

$$f(100) = f(111) = 11010$$

$$f(101) = f(110) = 00001$$

Simons Algorithm Analysis



$$|\pi_1\rangle = \frac{1}{\sqrt{2^n}} \sum_{x \in \Sigma^n} |0^m\rangle |x\rangle$$

$$|\pi_2\rangle = \frac{1}{\sqrt{2^n}} \sum_{x \in \Sigma^n} |f(x)\rangle |x\rangle$$

$$|\pi_3\rangle = \frac{1}{\sqrt{2^n}} \sum_{x \in \Sigma^n} |f(x)\rangle \otimes \left(\frac{1}{\sqrt{2^n}} \sum_{y \in \Sigma^n} (-1)^{x \cdot y} |y\rangle \right) = \frac{1}{2^n} \sum_{y \in \Sigma^n} \sum_{x \in \Sigma^n} (-1)^{x \cdot y} |f(x)\rangle |y\rangle$$

Simons Algorithm Analysis

$$\frac{1}{2^n} \sum_{y \in \Sigma^n} \sum_{x \in \Sigma^n} (-1)^{x \cdot y} |f(x)\rangle |y\rangle$$

$$\begin{aligned} p(y) &= \left\| \frac{1}{2^n} \sum_{x \in \Sigma^n} (-1)^{x \cdot y} |f(x)\rangle \right\|^2 \\ &= \left\| \frac{1}{2^n} \sum_{z \in \text{range}(f)} \left(\sum_{x \in f^{-1}(\{z\})} (-1)^{x \cdot y} \right) |z\rangle \right\|^2 \\ &= \frac{1}{2^{2n}} \sum_{z \in \text{range}(f)} \left| \sum_{x \in f^{-1}(\{z\})} (-1)^{x \cdot y} \right|^2 \end{aligned}$$

$$\text{range}(f) = \{f(x) : x \in \Sigma^n\}$$

$$f^{-1}(\{z\}) = \{x \in \Sigma^n : f(x) = z\}$$

Simons Algorithm Analysis

$$p(y) = \frac{1}{2^{2n}} \sum_{z \in \text{range}(f)} \left| \sum_{x \in f^{-1}(\{z\})} (-1)^{x \cdot y} \right|^2$$

Case 1: $s = 0^n$

Because f is a one-to-one, there a single element $x \in f^{-1}(\{z\})$ for every $z \in \text{range}(f)$:

$$\left| \sum_{x \in f^{-1}(\{z\})} (-1)^{x \cdot y} \right|^2 = 1$$

There are 2^n elements in $\text{range}(f)$, so

$$p(y) = \frac{1}{2^{2n}} \cdot 2^n = \frac{1}{2^n}$$

(for every $y \in \Sigma^n$).

Simons Algorithm Analysis

$$p(y) = \frac{1}{2^{2n}} \sum_{z \in \text{range}(f)} \left| \sum_{x \in f^{-1}(\{z\})} (-1)^{x \cdot y} \right|^2$$

Case 2: $s \neq 0^n$

There are two strings in the set $f^{-1}(\{z\})$ for each $z \in \text{range}(f)$; if $w \in f^{-1}(\{z\})$ either one of them, then $w \oplus s$ is the other.

$$\left| \sum_{x \in f^{-1}(\{z\})} (-1)^{x \cdot y} \right|^2 = \left| (-1)^{w \cdot y} + (-1)^{(w \oplus s) \cdot y} \right|^2 = \left| 1 + (-1)^{s \cdot y} \right|^2 = \begin{cases} 4 & s \cdot y = 0 \\ 0 & s \cdot y = 1 \end{cases}$$

There are 2^{n-1} elements in $\text{range}(f)$, so

$$p(y) = \frac{1}{2^{2n}} \sum_{z \in \text{range}(f)} \left| \sum_{x \in f^{-1}(\{z\})} (-1)^{x \cdot y} \right|^2 = \begin{cases} \frac{1}{2^{n-1}} & s \cdot y = 0 \\ 0 & s \cdot y = 1 \end{cases}$$

Classical post-processing

Running the circuit from Simon's algorithm one time gives us a random string $y \in \Sigma^n$.

Case 1: $s = 0^n$

$$p(y) = \frac{1}{2^n}$$

Case 2: $s \neq 0^n$

$$p(y) = \begin{cases} \frac{1}{2^{n-1}} & s \cdot y = 0 \\ 0 & s \cdot y = 1 \end{cases}$$

Suppose we run the circuit independently $k = n + r$ times, obtaining strings $y^1, \dots, y^k$.

$$\begin{array}{l} y^1 = y_{n-1}^1 \cdots y_0^1 \\ y^2 = y_{n-1}^2 \cdots y_0^2 \\ \vdots \\ y^k = y_{n-1}^k \cdots y_0^k \end{array} \quad M = \begin{pmatrix} y_{n-1}^1 & \cdots & y_0^1 \\ y_{n-1}^2 & \cdots & y_0^2 \\ \vdots & \ddots & \vdots \\ y_{n-1}^k & \cdots & y_0^k \end{pmatrix} \quad M \begin{pmatrix} s_{n-1} \\ \vdots \\ s_0 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ \vdots \\ 0 \end{pmatrix}$$

Using [Gaussian elimination](#) we can efficiently compute the [null space](#) (modulo 2) of M .
With probability greater than $1 - 2^{-r}$ it will be $\{0^n, s\}$.

Classical Difficulty

Any probabilistic algorithm making fewer than $2^{n/2-1} - 1$ queries will fail to solve Simon's problem with probability at least $1/2$.

- Simon's algorithm solves Simon's problem with a *linear* number of queries.
- Every classical algorithm for Simon's problem requires an *exponential* number of queries.

Lecture Summary: Key Concepts Covered I

- **The Query Model of Computation:** A framework where efficiency is measured by the number of oracle (black-box) queries.
- **Query Gates (O_f):** Unitary operations that allow a quantum computer to evaluate $f(x)$ without revealing its internal structure.
- **Examples of Query Problems:** Determining global properties (e.g., constant vs balanced) with minimal queries.
- **Deutsch's Problem:** Decide if $f : \{0, 1\} \rightarrow \{0, 1\}$ is constant or balanced.
- **Deutsch's Algorithm:** Solves the problem using **one query** (quantum speedup).
- **Phase Kickback:** Encodes function output into the phase, enabling interference-based computation.
- **Deutsch-Jozsa Problem:** Generalization to n -bit input functions.
- **Deutsch-Jozsa Algorithm:** Provides a deterministic solution in one query (exponential speedup).

Lecture Summary: Key Concepts Covered II

- **Bernstein-Vazirani Problem:** Efficiently determining a hidden bit string using quantum queries.
- **Simon's Problem:** Finding a hidden period (secret string) in a function with special structure.
- **Simon's Algorithm:** Quantum algorithm that solves Simon's problem exponentially faster than classical methods.
- **Simon's Algorithm Analysis:** Understanding interference and measurement outcomes.
- **Classical Post-Processing:** Extracting the secret string using linear algebra over $\mathbb{F}_2$.
- **Classical Difficulty:** Why classical algorithms require exponential time for these problems.